MODULE 4 PROGRAMMING THE COMPUTER

Unit 1 Computer Languages
Unit 2 Basic Principles of Computer Programming
Unit 3 Flowcharts and Algorithms

UNIT 1 COMPUTER LANGUAGES

CONTENTS

1.0 Introduction
2.0 Objectives
3.0 Main Content
    3.1 An Overview of Computer- Programming Language
    3.2 Types of Programmes, Language
        3.2.1 Machine Language
        3.2.2 Assembly (Low Level) Language
        3.2.3 High Level Language
        3.2.4 High Level Language
4.0 Conclusion
5.0 Summary
6.0 Tutor-Marked Assignment
7.0 References/Further Reading

7.0 INTRODUCTION

In this unit, we shall take a look at computer programming with emphasis on:

a. The overview of computer programming languages.
b. Evolutionary trends of computer programming languages.
c. Programming computers in a Beginner All-Purpose Symbolic Instruction Code (BASIC) language environment.

8.0 OBJECTIVES

At the end of this unit you should be able to provide background information about programming the computer.

3.0 MAIN CONTENT

3.1 An Overview of Computer Programming Languages

Basically, human beings cannot speak or write in computer language, and since computers cannot speak or write in human language, an intermediate language had to be devised to allow people to communicate with the computers. These intermediate languages, known as programming languages, allow a computer programmer to direct the activities of the computer. These languages are structured around a unique set of rules that dictate exactly how a programmer should direct the computer to perform a specific task. With the powers of reasoning and logic of human beings, there is the capability to accept an instruction and understand it in many different forms. Since a computer must be programmed to respond to specific instructions, instructions cannot be given in just any form. Programming languages standardise the instruction process. The rules of a particular language tell the programmer how the individual instructions must be structured and what sequence of words and symbols must be used to form an instruction.

- An operation code
- Some operands.

The operation code tells the computer what to do such as add, subtract, multiply and divide. The operands tell the computer the data items involved in the operations. The operands in an instruction may consist of the actual data that the computer may use to perform an operation, or the storage address of data. Consider for example the instruction: a = b + 5. The '=' and '+' are operation codes while 'a', 'b' and '5' are operands. The 'a' and 'b' are storage addresses of actual data while '5' is an actual data.

Some computers use many types of operation codes in their instruction format and may provide several methods for doing the same thing. Other computers use fewer operation codes, but have the capacity to perform more than one operation with a single instruction. There are four basic types of instructions, namely:

(a) input-output instructions;
(b) arithmetic instructions;
(c) branching instructions; and
(d) logic instructions.

An input instruction directs the computer to accept data from a specific input device and store it in a specific location in the store. An output

instruction tells the computer to move a piece of data from a computer storage location and record it on the output medium.

All of the basic arithmetic operations can be performed by the computer. Since arithmetic operations involve at least two numbers, an arithmetic operation must include at least two operands.

Branch instructions cause the computer to alter the sequence of execution of instruction within the program. There are two basic types of branch instructions; namely unconditional branch instruction and conditional branch instruction. An unconditional branch instruction or statement will cause the computer to branch to a statement regardless of the existing conditions. A conditional branch statement will cause the computer to branch to a statement only when certain conditions exist.

Logic instructions allow the computer to change the sequence of execution of instruction, depending on conditions built into the program by the programmer. Typical logic operations include: shift, compare and test.

3.2 Types of Programming Language

The effective utilisation and control of a computer system is primarily through the software of the system. We note that there are different types of software that can be used to direct the computer system. System software directs the internal operations of the computer, and applications software allows the programmer to use the computer to solve user made problems. The development of programming techniques has become as important to the advancement of computer science as the developments in hardware technology. More sophisticated programming techniques and a wider variety of programming languages have enabled computers to be used in an increasing number of applications.

Programming languages, the primary means of human-computer communication, have evolved from early stages where programmers entered instructions into the computer in a language similar to that used in the application. Computer programming languages can be classified into the following categories:

(a) Machine language
(b) Assembly language
(c) High level symbolic language
(d) Very high level symbolic language

3.2.1 Machine Language

The earliest forms of computer programming were carried out by using languages that were structured according to the computer stored data, that is, in a binary number system. Programmers had to construct programs that used instructions written in binary notation 1 and 0. Writing programs in this fashion is tedious, time-consuming and susceptible to errors.

Each instruction in a machine language program consists, as mentioned before, of two parts namely: operation code and operands. An added difficulty in machine language programming is that the operands of an instruction must tell the computer the storage address of the data to be processed. The programmer must designate storage locations for both instructions and data as part of the programming process. Furthermore, the programmer has to know the location of every switch and register that will be used in executing the program, and must control their functions by means of instructions in the program.

A machine language program allows the programmer to take advantage of all the features and capabilities of the computer system for which it was designed. It is also capable of producing the most efficient program as far as storage requirements and operating speeds are concerned. Few programmers today write applications programs in machine language. A machine language is computer dependent. Thus, an IBM machine language will not run on NCR machine, DEC machine or ICL machine. A machine language is the First Generation (computer) Language (IGL).

3.2.2 Assembly (Low Level) Language

Since machine language programming proved to be a difficult and tedious task, a symbolic way of expressing machine language instructions is devised. In assembly language, the operation code is expressed as a combination of letters rather than binary numbers, sometimes called mnemonics. This allows the programmer to remember the operations codes easily than when expressed strictly as binary numbers. The storage address or location of the operands is expressed as a symbol rather than the actual numeric address. After the computer has read the program, operations software are used to establish the actual locations for each piece of data used by the program. The most popular assembly language is the IBM Assembly Language.

Because the computer understands and executes only machine language programs, the assembly language program must be translated into a machine language. This is accomplished by using a system software program called an assembler. The assembler accepts an assembly

language program and produces a machine language program that the computer can actually execute. The schematic diagram of the translation process of the assembly language into the machine language is shown in fig.9.1. Although, assembly language programming offers an improvement over machine language programming, it is still an arduous task, requiring the programmer to write programs based on particular computer operation codes. An assembly language program developed and run on IBM computers would fail to run on ICL computers. Consequently, the portability of computer programs in a computer installation to another computer installation which houses different makes or types of computers were not possible. The low level languages are, generally, described as Second Generation (Computer) Language (2GL).

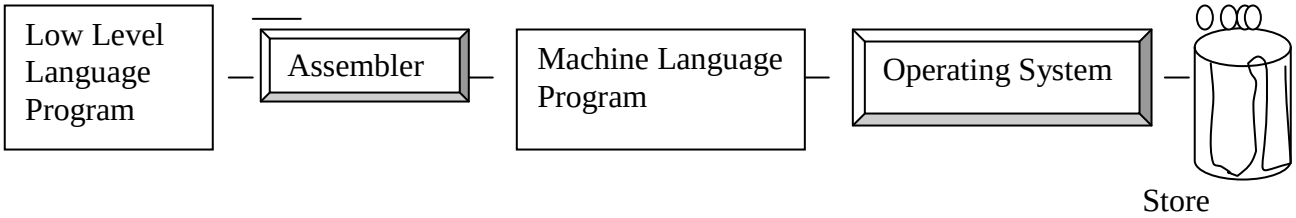


Fig.11: The assembly language program translation process

3.2.3 High Level Language

The difficulty of programming and the time required to program computers in assembly languages and machine languages led to the development of high-level languages. The symbolic languages, sometimes referred to as problem oriented languages reflect the type of problem being solved rather than the computer being used to solve it. Machine and assembly language programming is machine dependent but high level languages are machine independent, that is, a high-level language program can be run on a variety of computers.

While the flexibility of high level languages is greater than that of machine and assembly languages, there are close restrictions in exactly how instructions are to be formulated and written. Only a specific set of numbers, letters, and special characters may be used to write a high level program and special rules must be observed for punctuation. High level language instructions do resemble English language statements and the mathematical symbols used in ordinary mathematics. Among the existing and popular high level programming languages are Fortran, Basic, Cobol, Pascal, Algol, Ada and P1/1. The schematic diagram of the translation process of a high level language into the machine language is shown in fig.9.2. The high level languages are, generally, described as Third Generation (Computer) Language (3GL).

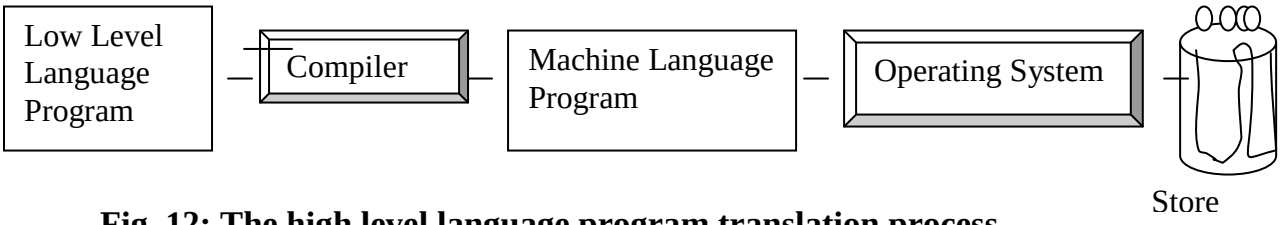


Fig. 12: The high level language program translation process

The general procedure for the compilation of a computer program coded in any high level language is conceptualised in Fig. 13.

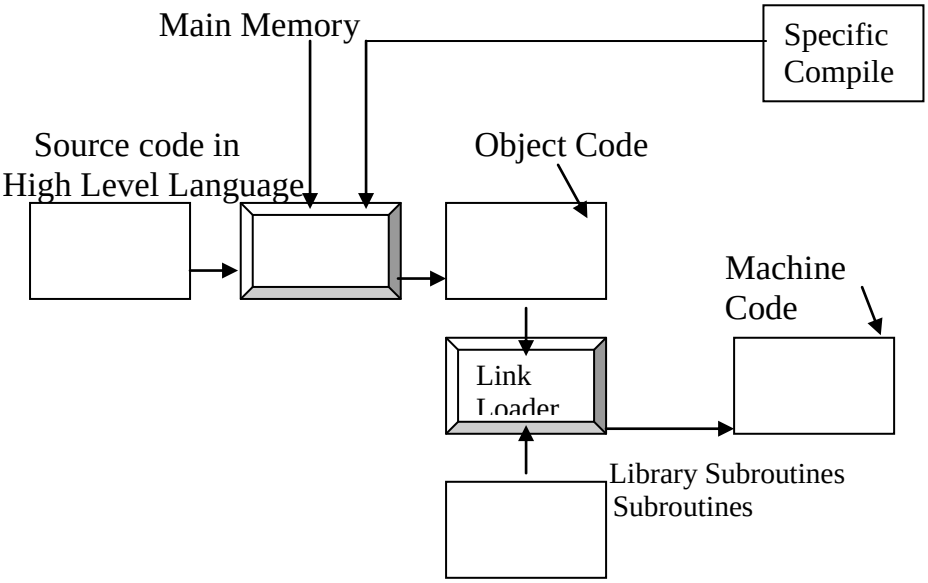


Fig. 13: General procedure for compiling a high level language program

3.2.4 Very High Level Language

Programming aids or programming tools are provided to help programmers do their programming work more easily. Examples of programming tools are:

- Program development systems that help users to learn programming, and to program in a powerful high level language. Using a computer screen (monitor) and keyboard under the direction of an interactive computer program, users are helped to construct application programs.
- A program generator or application generator that assists computer users to write their own programs by expanding simple statements into program code'.
- A database management system

- Debuggers that are programs that help the computer user to locate errors (bugs) in the application programs they write.

The very high level language generally described as the Fourth Generation (computer) Language (4GL), is an ill-defined term that refers to software intended to help computer users or computer programmers to develop their own application programs more quickly and cheaply. A 4GL, by using a menu system for example, allows users to specify what they require, rather than describe the procedures by which these requirements are met. The detailed procedure by which the requirements are met is done by the 4GL software which is transparent to the users.

A 4GL offers the user an English-like set of commands and simple control structures in which to specify general data processing or numerical operations. A program is translated into a conventional high-level language such as COBOL, which is passed to a compiler. A 4GL is, therefore, a non-procedural language. The program flows are not designed by the programmer but by the fourth generation software itself. Each user request is for a result rather than a procedure to obtain the result. The conceptual diagram of the translation process of very high level language to machine language is given Fig.14.

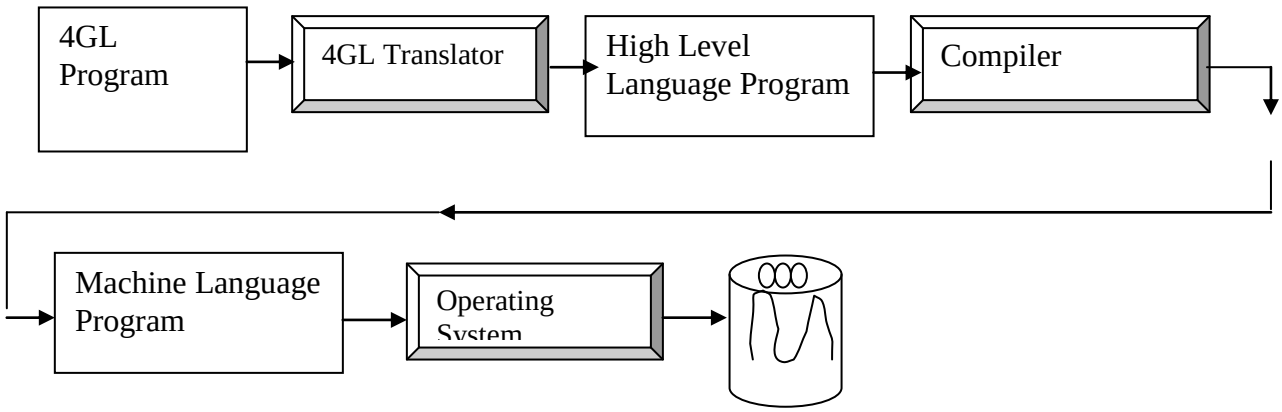


Fig. 14: The program translation process

The 4GL arose partly in response to the applications backlog. A great deal of programming time is spent maintaining and improving old programs rather than building new ones. Many organisations, therefore, have a backlog of applications waiting to be developed. 4GL, by stepping up the process of application design and by making it easier for end-users to build their own programs, helps to reduce the backlog.

4.0 CONCLUSION

Computer programming languages are means by which programmers manipulate the computer. The programming languages emanate from the need to program the computer in languages that would be easy for non-experts to understand and to reduce the enormity of tasks involved in writing programs in machine code. Programming languages have evolved from the machine language to assembly language, high level language and very high level programming language.

5.0 SUMMARY

We summarise the study of computer programming language as follows:

- Machine language is the binary language and it is made up of only 0s and 1s which represent the 'off' and 'on' stages of a computer's electrical circuits.

- Assembly language has a one-to-one relationship with machine language, but uses symbols and mnemonics for particular items. Assembly language, like machine language, is hardware specific, and is translated into machine language by an assembler.

- High level languages are usable on different machines and are designed for similar applications rather than similar hardware. They are procedural in that they describe the logical procedures needed to achieve a particular result. High level languages are translated into machine language by a compiler or an interpreter.

- In a high level language one specifies the logical procedures that have to be performed to achieve a result. In a fourth generation language, one needs to simply define the result one wants, and the requisite program instructions will be generated by the fourth generation software. Fourth generation languages are used in fourth generation systems in which a number of development tools are integrated in one environment.

6.0 TUTOR-MARKED ASSIGNMENT

1. What are computer programming languages?
2. Explain the following terms: machine language, source code, assembler, and compiler.

7.0 REFERENCES/FURTHER READING

Akinyokun, O.C. (1999). *Principles and Practice of Computing Technology*. Ibadan: International Publishers Limited.

Balogun, V.F., Daramola, O.A. Obe, O.O. Ojokoh, B.A., and Oluwadare S.A. (2006). *Introduction to Computing: A Practical Approach.* Akure: Tom- Ray Publications.

Francis Scheid (1983). *Schaum's Outline Series: Computers and Programming*. Singapore: McGraw-Hill Book Company.

Chuley, J.C. (1987). *Introduction to Low Level Programming for Microprocessors*. Macmillan Education Ltd.

Holmes, B.J. (1989). *Basic Programming*, (3rd ed.) ELBS.

UNIT 2 BASIC PRINCIPLES OF COMPUTER PROGRAMMING

CONTENTS

1.0 Introduction
2.0 Objectives
3.0 Main Content
    3.1 Problem Solving with the Computer
    3.2 Programming Methodology
    3.3 Stages of Programming
4.0 Conclusion
5.0 Summary
6.0 Tutor-Marked Assignment
7.0 References/Further Reading

1.0 INTRODUCTION

Computer programming is both an art and a science. In this unit you wil be exposed to some arts and science of computer programming, including principles of programming and stages of programming.

2.0 OBJECTIVES

At the end of this unit you should be able to:

- state the principles of computer programming
- explain the stages involved in writing computer programs.

3.0 MAIN CONTENT

3.1 Problem Solving With the Computer

The computer is a general-purpose machine with a remarkable ability to process information. It has many capabilities, and its specific function at any particular time is determined by the user. This depends on the program loaded into the computer memory being utilised by the user.

There are many types of computer programs. However, the programs designed to convert the general-purpose computer into a tool for a specific task or applications are called “Application programs”. These are developed by users to solve their peculiar data processing problems. Computer programming is the act of writing a program which a computer can execute to produce the desired result. A program is a series of instructions assembled to enable the computer to carry out a

specified procedure. A computer program is the sequence of simple instructions into which a given problem is reduced and which is in a form the computer can understand, either directly or after interpretation.

3.4 Programming Methodology

Principles of Good Programming

It is generally accepted that a good computer program should have the characteristics shown below:

- **Accuracy:** The program must do what it is supposed to do correctly and must meet the criteria laid down in its specification.
- **Reliability:** The program must always do what it is supposed to do, and never crash.
- **Efficiency:** Optimal utilisation of resources is essential. The program must use the available storage space and other resources in such as way that the system speed is not wasted.
- **Robustness:** The program should cope with invalid data and not stop without an indication of the cause of the source of error.
- **Usability:** The program must be easy enough to use and be well documented.
- **Maintainability:** The program must be easy to amend, having good structuring and documentation.
- **Readability:** The code of a program must be well laid out and explained with comments.

3.3 Stages of Programming

The preparation of a computer program involves a set of procedure. These steps can be classified into eight major stages, viz

(i) Problem definition
(ii) Devising the method of solution
(iii) Developing the method using suitable aids, e.g. pseudo code or flowchart.
(iv) Writing the instructions in a programming language
(v) Transcribing the instructions into “machine sensible” form
(vi) Debugging the program
(vii) Testing the program
(viii) Documenting all the work involved in producing the program.

(i) Problem definition

The first stage requires a good understanding of the problem. The programmer (i.e. the person writing the program) needs to thoroughly

understand what is required of a problem. A complete and precise unambiguous statement of the problem to be solved must be stated. This will entail the detailed specification which lays down the input, processes and output required.

(ii) Devising the method of solution

The second stage involved is spelling out the detailed algorithm. The use of a computer to solve problems (be it scientific or business data processing problems) requires that a procedure or an algorithm be developed for the computer to follow in solving the problem.

(iii) Developing the method of solution

There are several methods for representing or developing methods used in solving a problem. Examples of such methods are: algorithms, flowcharts, pseudo code, and decision tables.

(iv) Writing the instructions in a programming language

After outlining the method of solving the problem, a proper understanding of the syntax of the programming language to be used is necessary in order to write the series of instructions required to get the problem solved.

(v) Transcribing the instructions into machine sensible form

After the program is coded, it is converted into machine sensible form or machine language. There are some manufacturers written programs that translate users programs (source programs) into machine language (object code). These are called translators and instructions that machines can execute at a go, while interpreters accept a program and execute it line-by-line.

During translation, the translator carries out syntax check on the source program to detect errors that may arise from wrong use of the programming language.

(vi) Program debugging

A program seldomly executes successfully the first time. It normally contains a few errors (bugs). Debugging is the process of locating and correcting errors. There are three classes of errors.

- **Syntax errors:** Caused by mistake coding (illegal use of a feature of the programming language).

- **Logic errors:** Caused by faulty logic in the design of the program. The program will work but not as intended.

- **Execution errors:** The program works as intended but illegal input or other circumstances at run-time makes the program stop. There are two basic levels of debugging. The first level called desk checking or dry running is performed after the program has been coded and entered or key punched. Its purpose is to locate and remove as many logical and clerical errors as possible.

The program is then read (or loaded) into the computer and processed by a language translator. The function of the translator is to convert the program statements into the binary code of the computer called the object code. As part of the translation process, the program statements are examined to verify that they have been coded correctly, if errors are detected, a series of diagnostics referred to as an error message list is generated by the language translator. With this list in the hand of the programmer, the second level of debugging is reached.

The error message list helps the programmer to find the cause of errors and make the necessary corrections. At this point, the program may contain entering errors, as well as clerical errors or logic errors. The programming language manual will be very useful at this stage of program development.

After corrections have been made, the program is again read into the computer and again processed by the language translator. This is repeated over and over again until the program is error-free.

(vii) Program testing

The purpose of testing is to determine whether a program consistently produces correct or expected results. A program is normally tested by executing it with a given set of input data (called test data), for which correct results are known.

For effective testing of a program, the testing procedure is broken into three segments.

a. The program is tested with inputs that one would normally expect for an execution of the program.

b. Valid but slightly abnormal data is injected (used) to determine the capabilities of the program to cope with exceptions. For example, minimum and maximum values allowable for a sales-

amount field may be provided as input to verify that the program processed them correctly.

c. Invalid data is inserted to test the program's error-handling routines. If the result of the testing is not adequate, then minor logic errors still abound in the program. The programmer can use any of these three alternatives to locate the bugs.

Other methods of testing a program for correctness include:

- **Manual walk-through:** The programmer traces the processing steps manually to find the errors, pretending to be the computer, following the execution of each statement in the program, noting whether or not the expected results are produced.

- **Use of tracing routines:** If this is available for the language, this is similar to (1) above but is carried out by the computer; hence it takes less time and is not susceptible to human error.

- **Storage dump:** This is the printout of the contents of the computer's storage locations. By examining the contents of the various locations, when the program is halted, the instruction at which the program is halted can be determined. This is an important clue to finding the error that caused the halt.

- **Program documentation:** Documentation of the program should be developed at every stage of the programming cycle. The following are documentations that should be done for each program.

(a) Problem Definition Step

- A clear statement of the problem
- The objectives of the program (what the program is to accomplish)
- Source of request for the program.
- Person/official authorising the request

(b) Planning the Solution Step

- Flowchart, pseudo code or decision tables
- Program narrative
- Descriptive of input, and file formats

(c) Program source coding sheet

(d) User's manual to aid persons who are not familiar with the program to apply it correctly. The manual it contains a description of the program and what it is designed to achieve.

(e) Operator's manual to assist the computer operator to successfully run the program. This manual contains:

(i) Instructions about starting, running and terminating the program.
(ii) Message that may be printed on the console or VDU (terminal) and their meanings.
(iii) Setup and take down instruction for files.

Advantages of Program Documentation

a. It provides all necessary information for anyone who comes in contact with the program.
b. It helps the supervisor in determining the program's purpose, how long the program will be useful and future revision that may be necessary.
c. It simplifies program maintenance (revision or updating).
d. It provides information as to the use of the program to those unfamiliar with it.
e. It provides operating instructions to the computer operator.

4.0 CONCLUSION

The intelligence of a computer derives to a large extent from the quality of the programs. In this unit, we have attempted to present, in some detail, the principles and the stages involved in writing a good computer program.

5.0 SUMMARY

We have discussed the following:

- Principles of computer programming
- Stages of computer programming
- The interrelationship between different stages of programming.

6.0 TUTOR-MARKED ASSIGNMENT

1. Differentiate between program debugging and program testing.
2. What are the differences between syntax errors and logic errors? Give examples of each.
3. Is it possible to detect logic errors during program compilation? Explain the reason for your answer.

7.0 REFERENCES/FURTHER READING

Akinyokun, O.C. (1999). *Principles and Practice of Computing* Technology. Ibadan: International Publishers Limited.

Balogun, V.F., Daramola, O.A., Obe, O.O. Ojokoh, B.A. and Oluwadare S.A., (2006). *Introduction to Computing: A Practical Approach*. Akure: Tom-Ray Publications.

Francis Scheid (1983). *Schaum's Outline Series: Introduction to Computer Science.* Singapore: McGraw-Hill Book Company.

Holmes, B.J. (1989). *Basic Programming* (3rd ed.) ELBS.

Chuley, J.C. (1987). *Introduction to Low Level Programming for Microprocessors*. Macmillan Education Ltd.

UNIT 3 FLOWCHARTS AND ALGORITHMS

CONTENTS

1.0 Introduction
2.0 Objective
3.0 Main Content
    3.1 Flowcharts
    3.2 Flowchart Symbols
    3.3 Guidelines for Drawing
    3.4 Flowcharting the Problem
    3.5 Algorithms
    3.6 Pseudo Codes
    3.7 Decision Tables
4.0 Conclusion
5.0 Summary
6.0 Tutor-Marked Assignment
7.0 References/Further Reading

1.0 INTRODUCTION

This unit introduces to the principles of flowcharts and algorithms. The importance of these concepts is presented and the detailed steps and activities involved are also presented.

2.0 OBJECTIVE

At the end of this unit you should be able to:

- explain the principles of good programming ethics through flowcharting and algorithms.

3.0 MAIN CONTENT

3.1 Flowcharts

A flowchart is a graphical representation of the major steps of work in process. It displays in separate boxes the essential steps of the program and shows by means of arrows the directions of information flow. The boxes, most often referred to as illustrative symbols, may represent documents, machines or actions taken during the process. The area of concentration is on where or who does what, rather than on how it is done. A flowchart can also be said to be a graphical representation of an algorithm, that is, it is a visual picture which gives the steps of an algorithm and also the flow of control between the various steps.

3.2 Flowchart Symbols

Flowcharts are drawn with the help of symbols. The following are the most commonly used flowchart symbols and their functions:

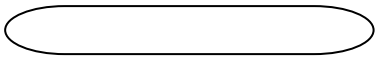
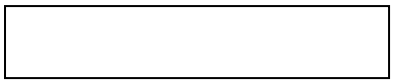
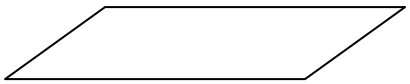
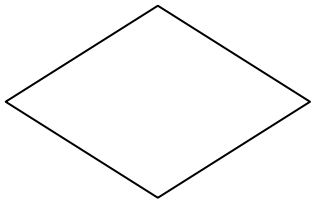
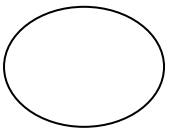
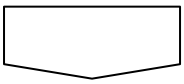
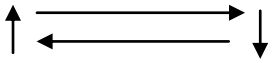
Symbols	Function
 **Terminator**	Used to show the START or STOP May show exit to a closed subroutine.
	Used for arithmetic calculations of process. E.g. Sum = X + Y + Z
	Used for Input and Output instructions, PRINT, READ, INPUT AND WRITE.
	Used for decision making. Has two or more lines leaving the box. These lines are labeled with different decision results, that is,' Yes', 'No', 'TRUE' or 'FALSE' or 'NEGATIVE' or 'ZERO'.
	Used for one or more named operations or program steps specified in a subroutine or another set of flowchart.
	Used for entry to or exit from another part of flowchart. A small circle identifies a junction point of the program
	Used for entry to or exit from a page
	Used to show the direction of travel. They are used in linking symbols. These show operations sequence and data flow directions.

Fig.15: Flowchart symbols and their functions

3.3 Guidelines for Drawing Flowcharts

- Each symbol denotes a type of operation: Input, Output, Processing, Decision, Transfer or Branch or Terminal.
- A note is written inside each symbol to indicate the specific function to be performed.
- Flowcharts are read from top to bottom.

- A sequence of operations is performed until a terminal symbol designates the end of the run or "branch" connector transfers control.

3.4 Flowcharting the Problem

The digital computer does not do any thinking and cannot make unplanned decisions. Every step of the problem has to be taken care of by the program. A problem which can be solved by a digital computer need not be described by an exact mathematical equation, but it does need a certain set of rules that the computer can follow. If a problem needs intuition or guessing, or is so badly defined that it is hard to put into words, the computer cannot solve it. You have to define the problem and set it up for the computer in such a way that every possible alternative is taken care of. A typical flowchart consists of special boxes, in which are written the activities or operations for the solution of the problem. The boxes, linked by means of arrows, show the sequence of operations. The flowchart acts as an aid to the programmer, who follows the flowchart design to write his programs.

3.5 Algorithms

Before a computer can be put to any meaningful use, the user must be able to come out with or define a unit sequence of operations or activities (logically ordered) which gives an unambiguous method of solving a problem or finding out that no solution exists. Such a set of operations is known as an ALGORITHM.

Definition: An algorithm, named after the ninth century scholar Abu Jafar Muhammad Ibn Musu Al-Khowarizmi, is defined as follows:

- An algorithm is a set of rules for carrying out calculations either by hand or a machine.
- An algorithm is a finite step-by-step procedure to achieve a required result.
- An algorithm is a sequence of computational steps that transform the input into the output.
- An algorithm is a sequence of operations performed on data that have to be organised in data structures.
- An algorithm is an abstraction of a program to be executed on a physical machine (model of computation).

The most famous algorithm in history dates well before the time of the ancient Greeks: this is Euclids algorithm for calculating the greatest common divisor of two integers. Before we go into some otherwise

complex algorithms, let us consider one of the simplest but common algorithms that we encounter everyday.

The classic multiplication algorithm

For example to multiply 981 by 1234, this can be done using two methods (algorithms) viz:

a. Multiplication the American way:

Multiply the multiplicand one after another by each digit of the multiplier taken from right to left.

```
      981
      1234
  ____________
      3924
       2943
      1962
      981
      1210554
  ____________
```

b. Multiplication , the British way:

Multiply the multiplicand one after another by each digit of the multiplier taken from left to right.

```
      981
      1234
  ____________
      981
      1962
       2943
      3924
      1210554
  ____________
```

An algorithm therefore can be characterised by the following:

(i) A finite set or sequence of actions
(ii) This sequence of actions has a unique initial action
(iii) Each action in the sequence has unique successor
(iv) The sequence terminates with either a solution or a statement that the problem is unresolved.

An algorithm can therefore be seen as a step-by-step method of solving a problem.

Examples

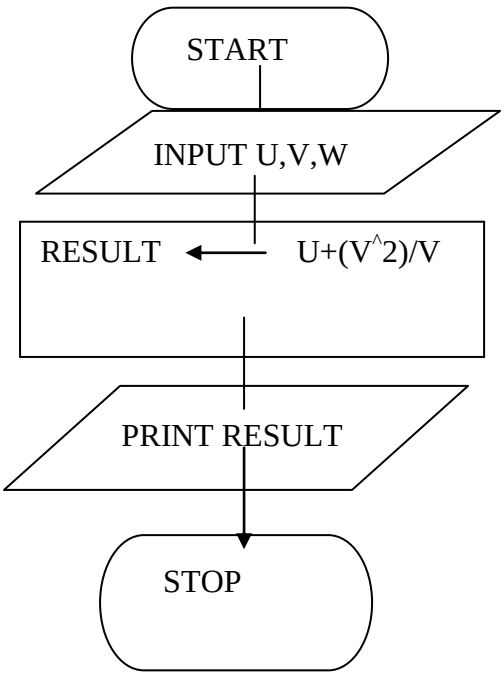
1. Write an algorithm to read values for three variables. U, V, and W and find a value for RESULT from the formula: RESULT = U + V^2/W. Draw the flowchart.

Solution

Algorithm

(i) Input values for U, V, and W
(ii) Computer value for result
(iii) Print value of result
(iv) Stop

Flowchart



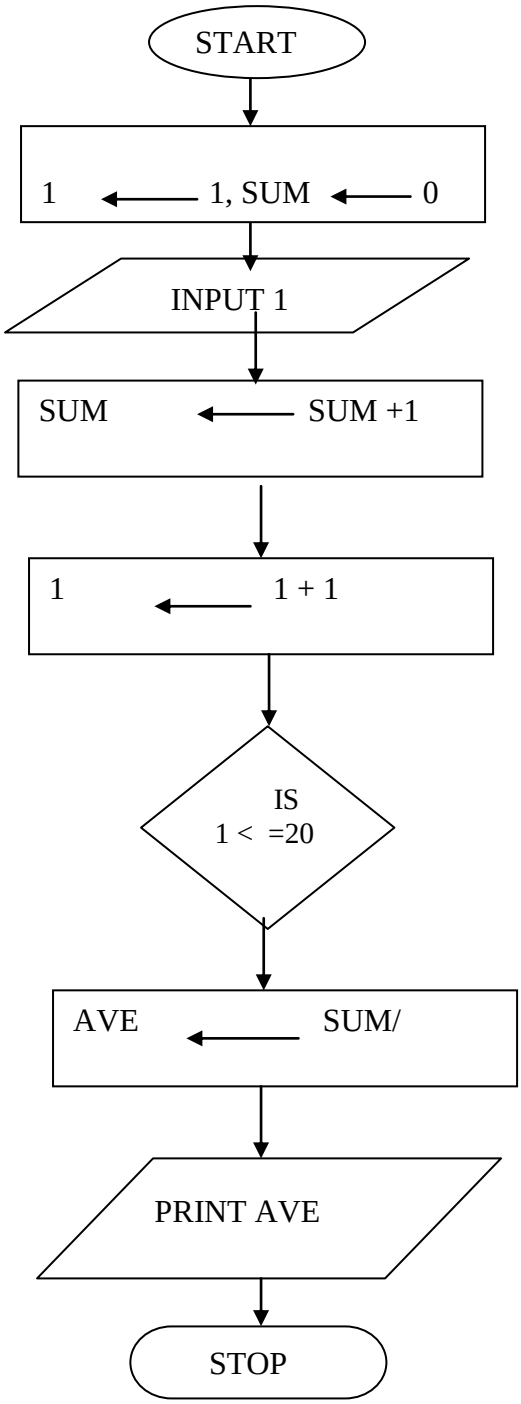
2. Suppose you are given 20 numbers. Prepare the algorithm that adds up these numbers and find the average. Draw the flowchart.

Solution

Algorithm

- Set up a Counter (1) which counts the number of times the loop is executed. Initialise Counter (1) to 1.
- Initialize sum to zero.
- Input value and add to sum.

- Increment the counter (1) by 1.
- Check how many times you have added up the number, if it is not up to the required number of times, to step (iii).
- Compute the average of the numbers.
- Print the average.
- Stop.



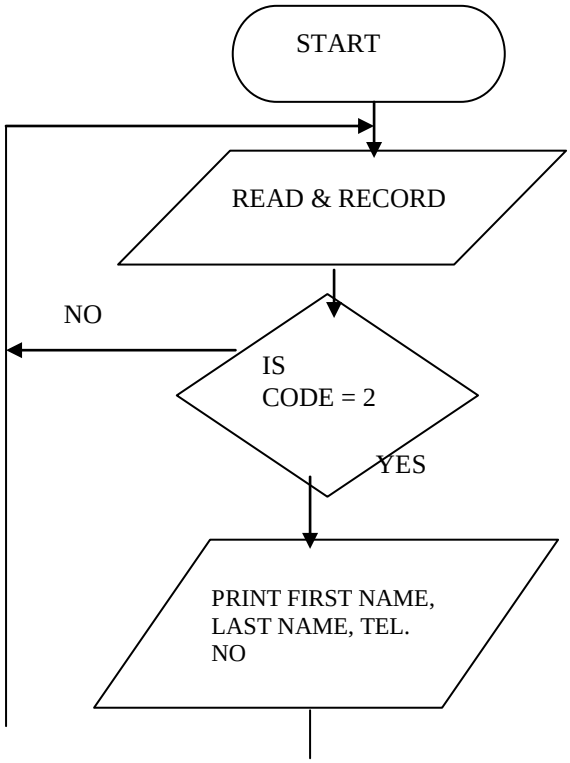
3. Prepare an algorithm that indicates the logic for printing the name and telephone number for each female in a file (Code field is 2 for female). Draw the flowchart.

Solution

Algorithm

(i) Read a record
(ii) Determine if the record pertains to a female (that is, determine if the code field is equal to 2).
(iii) If the code field is not equal to 2, then do not process this record any further, since it contains data for a male. Instead, read the next record; that is, go back to step (i).
(iv) If the record contains data for a female (that is, code is equal to 2), then print out the following fields: first name, last name, telephone number.
(v) Go back to step (i) to read the next record.

Flowchart



Note: Nothing indicates the end of records processing here.

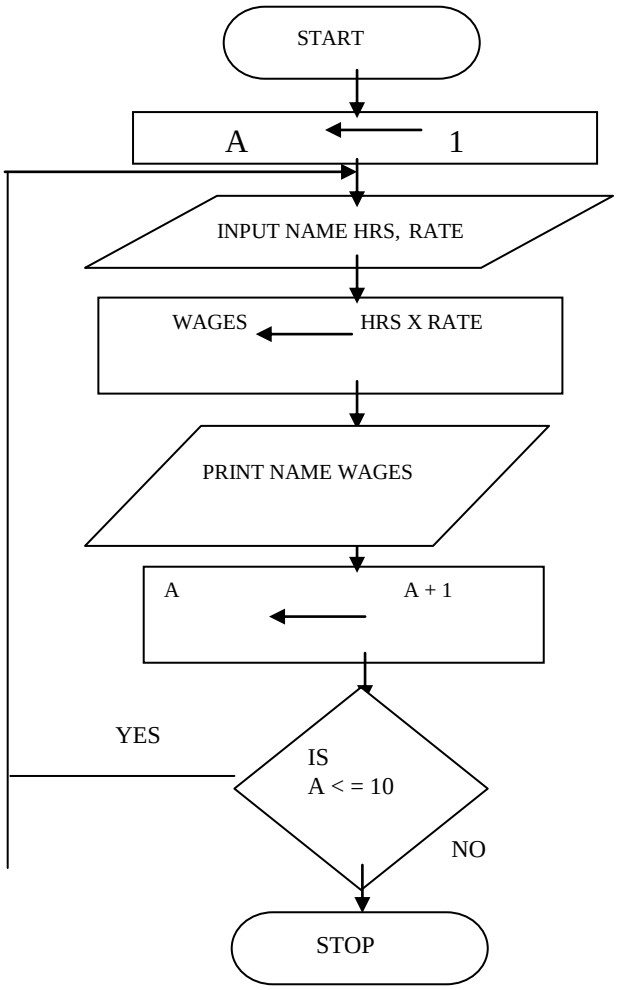
4. Prepare an algorithm that prints name and weekly wages for each employee out of 10 where name, hours worked, and hourly rate are read in. Draw the flowchart.

Solution

Algorithm

(i) Initialise Counter (A) to 1
(ii) Read name, hours and rate and number of workers
(iii) Let the wage be assigned the product of hours and rate
(iv) Print name and wages.
(v) Increment the counter (A) by 1.
(vi) Make a decision (Check how many times you have calculated the wages).
(vii) Stop processing, if you have done it the required number of times.

Flowchart



3.6 Pseudocodes

A pseudocode is a program design aid that serves the function of a flowchart in expressing the detailed logic of a program. Sometimes a program flowchart might be inadequate for expressing the control flow and logic of a program. By using Pseudocodes, program algorithms can be expressed as English-language statements. These statements can be used both as a guide when coding the program in a specific language and as documentation for review by others. Because there is no rigid rule for constructing pseudocodes, the logic of the program can be expressed in a manner that does not conform to any particular programming language. A series of structured words is used to express the major program functions. These structured words are the basis for writing programs using a technical term called "structure programming".

Example

Construct a pseudocode for the problem in the example above.

```
BEGIN
STORE 0 TO SUM
STORE 1 TO COUNT
            DO WHILE COUNT not greater than 10
            ADD COUNT to SUM
      INCREMENT COUNT by 1
      ENDWILE
      END
```

3.7 Decision Tables

Decision tables are used to analyse a problem. The conditions applying in the problem are set out and the actions to be taken, as a result of any combination of the conditions arising are shown. They are prepared in conjunction with or in place of flowcharts. Decision tables are a simple yet powerful and unambiguous way of showing the actions to be taken when a given set of conditions occur. Moreover, they can be used to verify that all conditions have been properly catered for. In this way they can reduce the possibility that rare or unforeseen combinations of conditions will result in confusion about the actions to be taken.

Decision tables have standardised formats and comprise of four sections.

(a) Conditions Stub: This section contains a list of all possible conditions which could apply in a particular problem.

(b) Conditions Entry: This section contains the different combination of the conditions, each combination being given a number termed a 'Rule'.

(c) Action Stub: This section contains a list of the possible actions which could apply for any given combinations of conditions.

(d) Action Entry: This section shows the actions to be taken for each combination of conditions. **Writing the instructions in a programming language (program coding)**

The instructions contained in the algorithm must be communicated to the computer in a language it will understand before it can execute them. The first step is writing these instructions in a programming language (program coding). Program coding is the process of translating the planned solution to the problems, depicted in a flowchart, pseudocode or decision table, into statements of the program. The program flowchart, pseudocode decision table as the case may be, is as a guide by the programmer as he describes the logic in the medium of a programming language. The coding is usually done on coding sheets or coding forms.

4.0 CONCLUSION

Flowcharts, decision tables, pseudocodes and algorithms are essential ingredients to the writing of good programs. If they are done properly they lead to a reduction in errors in programs. They help minimise the time spent in debugging. In addition, they make logic errors easier to trace and discover.

5.0 SUMMARY

This unit teaches that:

a. A flowchart is a graphical representation of the major steps of work in process. It displays in separate boxes the essential steps of the program and shows by means of arrows the directions of information flow.
b. Decision tables are used to analyse a problem. The conditions applying in the problem are set out and the actions to be taken, as a result of any combination of the conditions arising, are shown.
c. A pseudocode is a program design aid that serves the function of a flowchart in expressing the detailed logic of a program.
d. An algorithm is a set of rules for carrying out calculation either by hand or a machine.